P1293R2
Revision of P1293R1
2019-01-21
Mike Spertus, Symantec
mike_spertus@symantec.com
Nathan Wilson
nwilson20@gmail.com
Target: Library Evolution, Library

`ostream_joiner`

Status update

This paper provides wording for merging into C++20 `ostream_joiner` with the CTAD-related changes below. In addition, we have updated the specification as suggested by Casey Carter to meet the requirements of C++20 output iterators as described below.

The main purpose of assigning this revision to LEWG is to clarify LEWG polling from San Diego

The following LEWG straw polls from San Diego provide support for merging `ostream_joiner` in C++20

> We are interested in fixing the constructor/CTAD/make_ostream_joiner.

SF	F	N	A	SA
5	6	0	0	0

> We want to prioritize ostream joiner as per the TS + above ctor change for C++20 (No later than Kona)

SF	F	N	A	SA
3	6	2	1	1

However, note that there was a previous poll

> Explore a solution in a post-ranges design (do nothing for C++20).

SF	F	N	A	SA
2	8	5	0	1

Based on the above, we propose to merge as above but continue to explore the design space post-ranges.

Summary

This paper updates the `ostream_joiner` proposal to reflect experience in Library Fundamentals V2 and changes in the core language since then. Specifically, we add a deduction guide to ensure that `ostream_joiner` is constructed correctly from a C string literal and remove the now-redundant `make_ostream_joiner`.

Deduction guides

With the addition of class template argument deduction in C++17, it seems desirable to get rid of the clumsy `make_ostream_joiner` as we did with class templates like `boyer_moore_searcher`. However, as Nathan Myers has pointed out, without any deduction guide, the decision to take the delimiter by reference means that

```
ostream_joiner j(cout, ", ");
```

produces a compile time error as the delimiter type is deduced as `char const[3]`. To fix this, add the following to providing the following deduction guide

```
template<class DelimT, class CharT, class Traits>
ostream_joiner(basic_ostream<CharT, Traits>&, DelimT) -> ostream_joiner<DelimT, CharT, Traits>;
```

C++20 iterator requirements

C++20 output iterators are required to be default constructible, so the wording below adds a (useless) default constructor and requires the delimiter type to be Semiregular at Casey Carter's suggestion.

Wording

Based on the above, we propose applying the following wording to the C++20 working draft. Modifications to Library Fundamentals TS v2 are as given in P1293.

Modify iterator.synopsis as follows

```
template<class charT, class traits = char_traits<charT>>
  class ostreambuf_iterator;

template <class DelimT, class charT = char, class traits = char_traits<charT>>
  class ostream_joiner;
```

Add a subsection before iterator.range as follows

22.6.5 Class template `ostream_joiner` iterator.ostream.joiner

`ostream_joiner` writes (using `operator<<`) successive elements onto the output stream from which it was constructed. The delimiter that it was constructed with is written to the stream between every two `T`s that are written. It is not possible to get a value out of the output iterator. Its only use is as an output iterator in situations like

```
while (first != last)
  *result++ = *first++;
```

`ostream_joiner` is defined as

```
template <class DelimT, class charT = char, class traits = char_traits<charT>>
  requires Semiregular<DelimT>
class ostream_joiner {
public:
  using char_type = charT;
  using traits_type = traits;
  using ostream_type = basic_ostream<charT, traits>;
  using iterator_category = output_iterator_tag;
  using value_type = void;
  using difference_type = ptrdiff_t;
  using pointer = void;
  using reference = void;

  ostream_joiner() = default;
  ostream_joiner(ostream_type& s, const DelimT& delimiter);
  ostream_joiner(ostream_type& s, DelimT&& delimiter);
  template<typename T>
  ostream_joiner& operator=(const T& value);
  ostream_joiner& operator*() noexcept;
  ostream_joiner& operator++() noexcept;
  ostream_joiner& operator++(int) noexcept;
private:
  ostream_type* out_stream = nullptr; // exposition only
  DelimT delim;             // exposition only
  bool first_element = true;       // exposition only
};

template<class DelimT, class CharT, class Traits>
ostream_joiner(basic_ostream<CharT, Traits>&, DelimT) -> ostream_joiner<DelimT, CharT, Traits>;
```

22.6.5.1 `ostream_joiner` constructor iterator.ostream.joiner.cons

```
ostream_joiner(ostream_type& s, const DelimT& delimiter);
```

Effects: Initializes `out_stream` with `addressof(s)`, `delim` with `delimiter`, and `first_element` with `true`.

```
ostream_joiner(ostream_type& s, DelimT&& delimiter);
```

Effects: Initializes `out_stream` with `addressof(s)`, `delim` with `std::move(delimiter)`, and `first_element` with `true`.

22.6.5.2 `ostream_joiner` operations iterator.ostream.joiner.ops

```
template<typename T>
ostream_joiner& operator=(const T& value);
```

Effects:

```
  if (!first_element)
    *out_stream << delim;
  first_element = false;
  *out_stream << value;
return *this;
```

```
ostream_joiner& operator*() noexcept;
```

Returns: `*this`.

```
ostream_joiner& operator++() noexcept;
ostream_joiner& operator++(int) noexcept;
```

Returns: `*this`.